

Isolation and Protection in Operating Systems

3. System Calls in x86-64

Prof. Dr. Michael Schöttner

Dr. Fabian Ruhland

- Until now all user mode threads (Ring 3) can just call all kernel functions directly
 - The CPU triggers an General Protection Fault (GPF) if a privileged instruction is used inside one of the called kernel function
 - Usually, kernel code is protected from direct execution in ring 3 via paging
- We now want to implement *System Calls*, which expose kernel functions for indirect and controlled execution in ring 3

■ Call Gate

- Descriptor in the GDT
- Access via a `call` or `jmp` instruction
- Stack is switched via the TSS

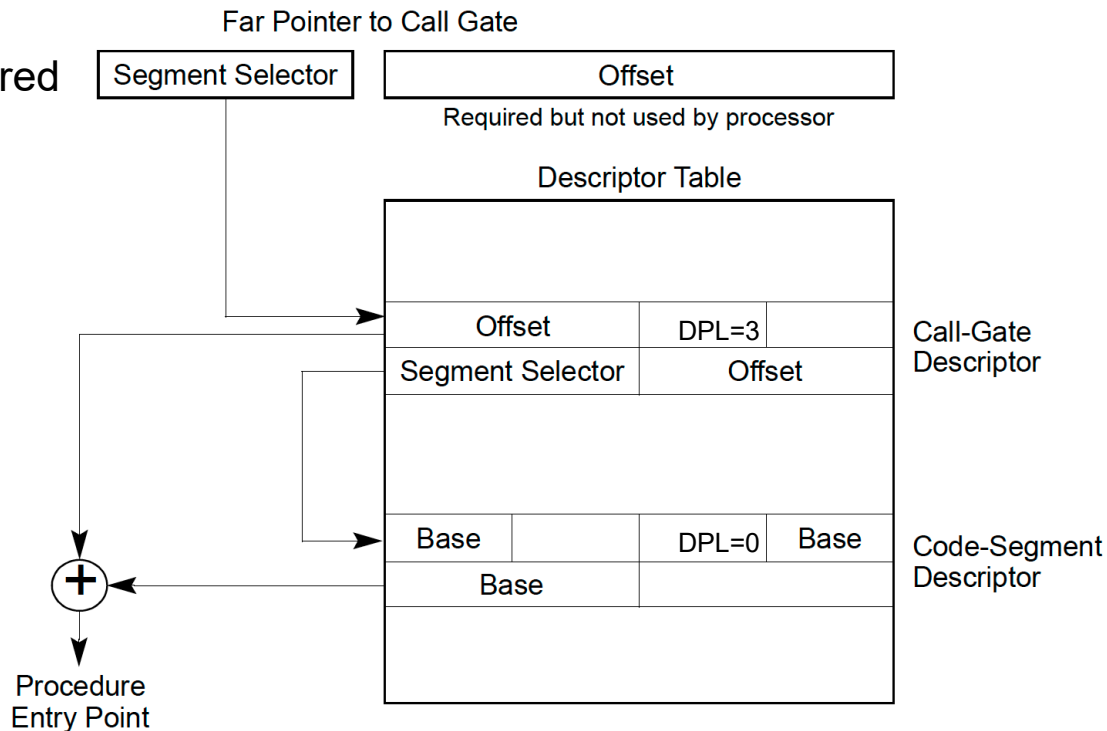
■ Interrupt Trap Gate

- Descriptor in the IDT
- Access via a `int` instruction
- Stack is switched via the TSS

■ Fast System Calls

- Configured through MSR (Model Specific Register)
- Access via a `syscall/sysret` instruction
- Stack must be switched manually by the operating system

- Offset in register and code segment descriptor are ignored



- Interrupt/Trap Gate**

The diagram illustrates the structure of the Interrupt/Trap Gate, which is 32 bits wide. The fields are as follows:

 - Reserved:** Bits 31 down to 12 (20 bits).
 - Offset 63..32:** Bits 31 down to 8 (24 bits).
 - Offset 31..16:** Bits 31 down to 16 (16 bits). This section is further divided into:
 - Offset 31..16 (bits 31-16)
 - P (bit 15)
 - DPL (bits 14-13)
 - 0 (bit 12)
 - TYPE (bits 11-8)
 - 0 0 0 (bits 7-5)
 - 0 0 (bits 4-3)
 - IST (bit 2)
 - Segment Selector and Offset 15..0:** Bits 31 down to 0 (32 bits). This section is further divided into:
 - Segment Selector (bits 31-16)
 - Offset 15..0 (bits 15-0)

Interrupt Stack Table

■ Interrupt Gate

- *Interrupt Enable* flag is deleted
- Used for interrupt handlers → sequential execution (on single core CPUs)

■ Trap Gate

- *Interrupt Enable* flag is not deleted
- Used for system calls

■ External/Hardware interrupts

- Triggered by a device (e.g., timer)
- Come from the outside (from the CPU's point of view)

■ Internal/Software interrupts

- Triggered by the CPU itself
 - Exceptions (see next slides)
 - Assembly instruction `INT <nr>`
 - Used for *System Calls* (Linux, Windows NT, MacOS, MSDOS)

■ Fault

- Operating system can handle and recover from faults (e.g., Page Fault)
- CPU state and the address of the fault triggering instruction are stored

■ Trap

- Triggered by a special instruction (e.g., INT 3 - Breakpoint)
- Program can be continued

■ Abort

- Unrecoverable error
- Triggers are hard to localize exactly
- Cause a reboot (e.g., Double Fault)

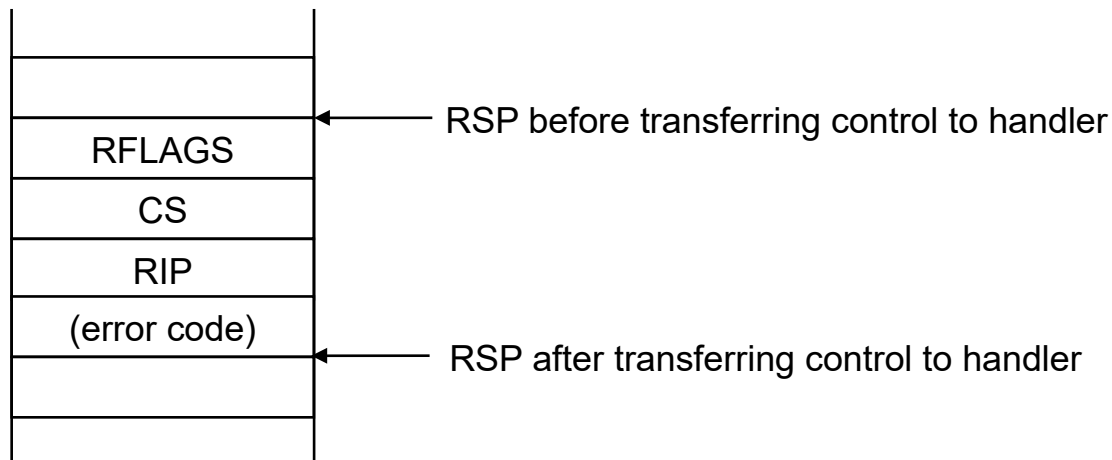
- Interrupts and exceptions are identified via a *Vector Number*
- 0 – 31 are reserved for exceptions
- 32 – 255 are freely available
- Examples for exceptions:

Vector	Exception	Vector	Exception
0	Division by 0	11	Segment not present
1	Debug	12	Stack overflow
2	NMI	13	General protection fault
3	Break	14	Page fault
4	Overflow	16	-
5	Bounds range exceeded	18	Machine check
6	Illegal instruction	...	...
8	Double fault		

Stackup Buildup without Ring Change

- If the privilege level is not changed, the CPU does not switch the stack
- The interrupt handler operates on the stack of the interrupted thread
- This happens in HeineOS (all our code runs in ring 0)
- Only some exceptions push an error code

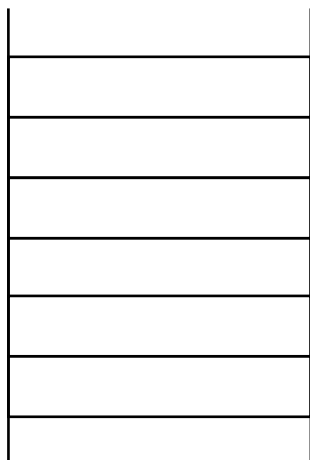
Stack of the interrupted thread



Stack Buildup with Ring Change

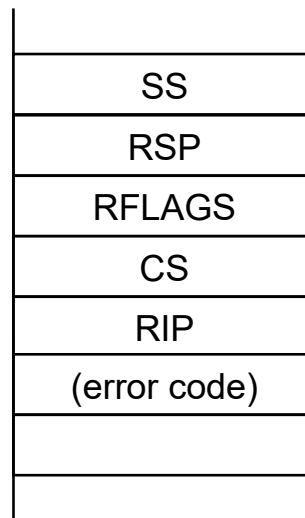
- New stack is read from *Task State Segment* (see last chapter)

Stack of the interrupted thread



RSP before transferring
control to handler

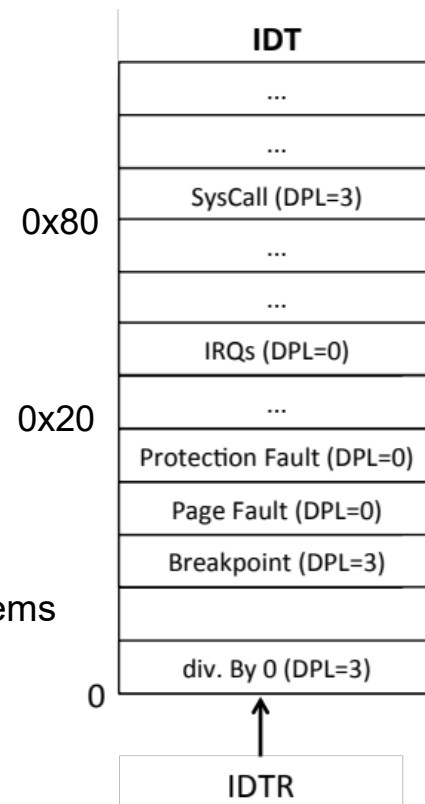
Handler Stack



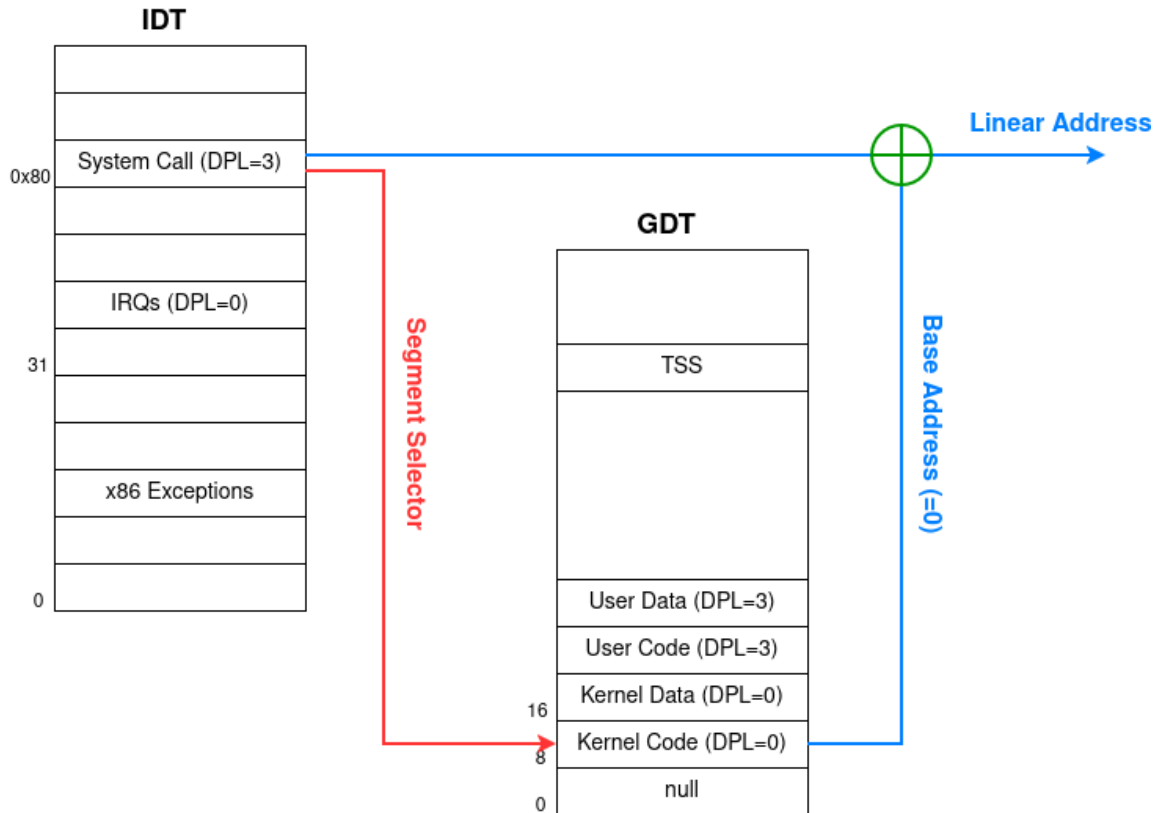
RSP after transferring
control to handler

IDT: Typical Example

- Entries 0-31 are reserved for x86 CPU exceptions
 - Either DPL=0 or DPL=3, depending on use case
- Entries 32-47 for external interrupts
 - IRQs have DPL=0
 - IRQ 0 != Vector 0
- System calls via trap gate (entry 0x80)
 - DPL=3
 - Replaced by SYSENTER/SYSEXIT in modern operating systems
→ faster, avoids interrupt overhead



System Call via Trap Gate



Fast System Calls (syscall/sysret)

- Use Model Specific Registers (MSRs)
- Require specific GDT layout
- Stack must be switched manually in syscall handler
- Use FS und GS registers
 - Only base address is used
 - Typically the GS base address points to the kernel stack

